Universität Zürich UZH

GEO877

Spatial Algorithm

Introduction

Inhye Kong

Target audience

- BSc/MSc students working on major/minor in Geography with previous programming experiences (e.g. GEO876)
- It is assumed that you understand the basics of programming – variables, functions, iterations, objective oriented programming, and are familiar with writing python codes
- You are interested in understanding how algorithms work – not just pressing a button on GUI (graphical user interface) in ArcGIS/QGIS or running off-the-shelf packages (Geopandas, Shapely) in python

Learning objectives

"computational geometry"

- Understand spatial relations and attributes in operating spatial algorithms
- Define spatial questions and explore possible solutions
- Implement computer language (python*) to operationalize solutions
- Critical thinking to discuss the accuracy and efficiency of solutions

* 4C in higher education in the 21st century: critical thinking, communication, collaboration, and creativity

Algorithms for geographic questions

- Algorithms underpin how the task is executed
- Given the hint “computational geometry”, what do you think the spatial questions in this course will be about?
- Extra hint: what are the geographic elements in spatial analysis?

Basic elements of (spatial) data

- **Point** in coordinates (x,y) assuming Cartesian space
- **PointGroup** as a collection of points
- **Segment** as a finite line with a start and an end point
  - Segments are bounded at both ends
  - Rays have a start but no end
  - Lines have neither a start nor an end
- **Polygon** with an ordered set of point coordinates

- We will not cover raster, but can be implemented for your own code if necessary

Spatial Algorithms in GE0877

01

Distance

02

Centroid and area

03

Segment: buffer and intersection

04

Points in polygon

05

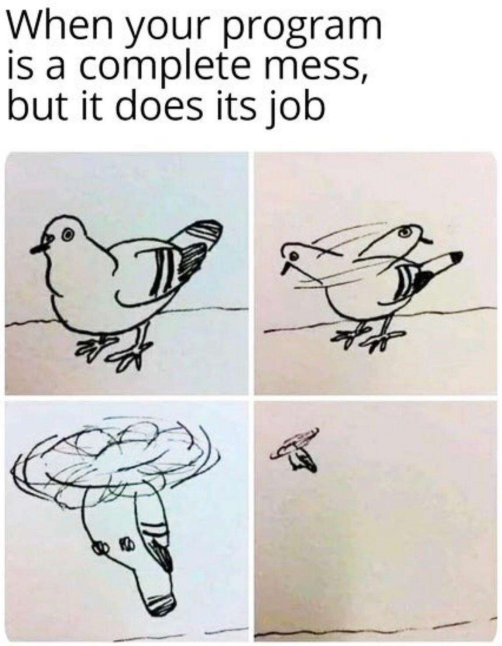
Polygon around points

06

Grid-based spatial indexing
(Shortest path routing)

Why do we (still) care about algorithms in the era of GenAI?

- **GenAI** should be a helpful tool in this course to translate your thoughts into actionable codes; and you should be able to get familiar with, and become skillful in using it in an era of AI
- But it is important that you do not lose your grip and aware of what you are building…!

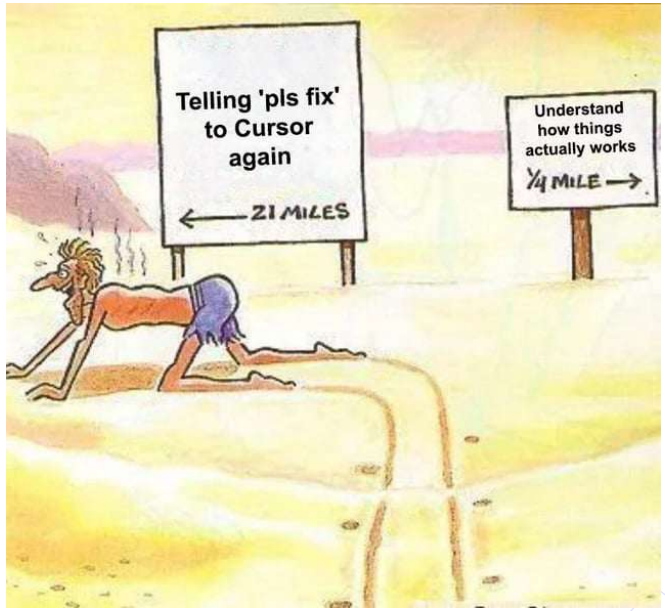


https://www.reddit.com/r/ProgrammerHumor/comments/f4dbmj/saw_this_on_dank_memes/

Careful…

- "vibe coding" may take you to endless bugs and get you lost (the models get better and there are some structural design techniques to avoid this but)

- This course is designed to 1) think about algorithms behind the GUIs or off-the-shelf packages for spatial questions, 2) try critical thinking of accuracy and efficiency of possible solutions and 3) implement your own ideas for possible solutions using python



https://www.reddit.com/r/ProgrammerHumor/comments/1jilyj1/vibecoding/

Course overview

Week	Date	Content
Week 1	19-Feb	Introduction
Week 2	26-Feb	Lecture 1
Week 3	5-Mar	Lecture 2
Week 4	12-Mar	Lecture 3
Week 5	19-Mar	paper review presentation, Lecture 4
Week 6	26-Mar	paper review presentation
Week 7	2-Apr	group work
Week 8	9-Apr	No class; Easter break
Week 9	16-Apr	Lecture 5
Week 10	23-Apr	Lecture 6
Week 11	30-Apr	progress presentation
Week 12	7-May	group work (on-demand feedback)
Week 13	14-May	group work (on-demand feedback)
Week 14	21-May	final presentation
Week 15	28-May	final presentation
	11-Jun	Deadline for written report

Course overview: lectures

Active participation and interactive communication are key to take the best out of this course

I present	Main concepts of spatial questions
We together	Brainstorm how the questions can be solved
I present	Conceptual approaches and pseudo codes
We together	Talk about accuracy, efficiency, and alternate approaches
I present	Examples of code execution in python

Course overview: group work

- After this class, you can browse and start to choose a paper; your teammates will be announced at the end of Lecture 1 (Feb 26)
- As a group, you will work together on the chosen paper, to find tractable and relevant questions from the paper and "replicate" the result(s) using your own algorithmic application
- As a group, you will make three (short) presentations and one final written report
- Grading is based on group work; it will be of absolute grading, thus inter-group communication is highly encouraged – some of you might be struggling with the same spatial questions

Assessment details

Your group project will be assessed based on:

- Paper review presentation (10%)
- Progress presentation will not be graded
- Final presentation (20%)
- Written final report (70%)

  summary of your project; three key algorithms; code details

*Where you work – PyCharm, VSCode, Jupyter Notebook, Google Collab etc – is your choice; but you should work together with your teammates, and the final code should be handed out in Jupyter Notebook

Communication

- @Teams
  - All materials, including lecture slides and papers
  - In-group communication at private channel (I will create for you once the group is defined)
  - General board for open communications

- @UZH storage for course directory
  - Final submission

Useful resources

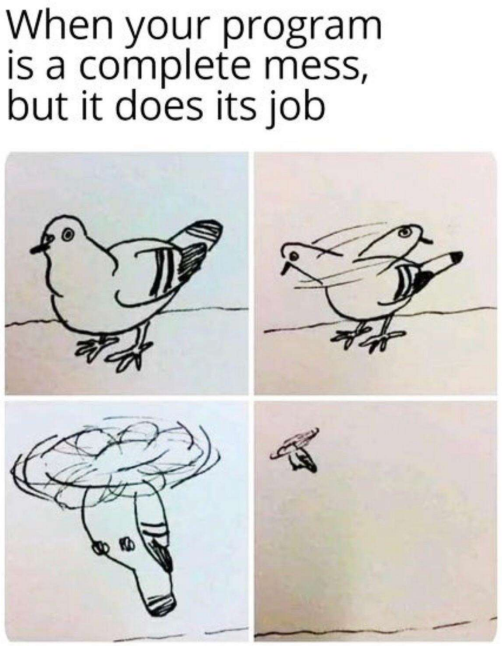
You may get helped with…

- Ningchuan Xiao. GIS Algorithms. (digital version available in library)
- Dorman, M., Graser, A., Nowosad, J., Lovelace, R. (2025). Geocomputation with Python. https://py.geocompx.org/
- Tenkanen, H., Heikinheimo, V. & Whipp, D. (2022). Introduction to Python for Geographic Data Analysis (https://pythongis.org)
- Rey, S., Arribas-Bel, D., & Wolf, L.J (2020). Geographic Data Science with Python (https://geographicdata.science/book/intro.html#)

Your GenAI, Stack Overflow and your colleagues!

Good algorithms?

- In this course, we care about good algorithms, i.e. algorithms that are both **correct** and **efficient**
- How can we tell if the algorithm is correct and efficient?
- Correct? some are easy to answer, but some are difficult – unforeseen cases to lead to failure
- Efficient? Theoretical limit on how it can be efficient, so mostly about a relative comparison



https://www.reddit.com/r/ProgrammerHumor/comments/f4dbmj/saw_this_on_dank_memes/

GE0876 recap:

Looking for a tree of interest

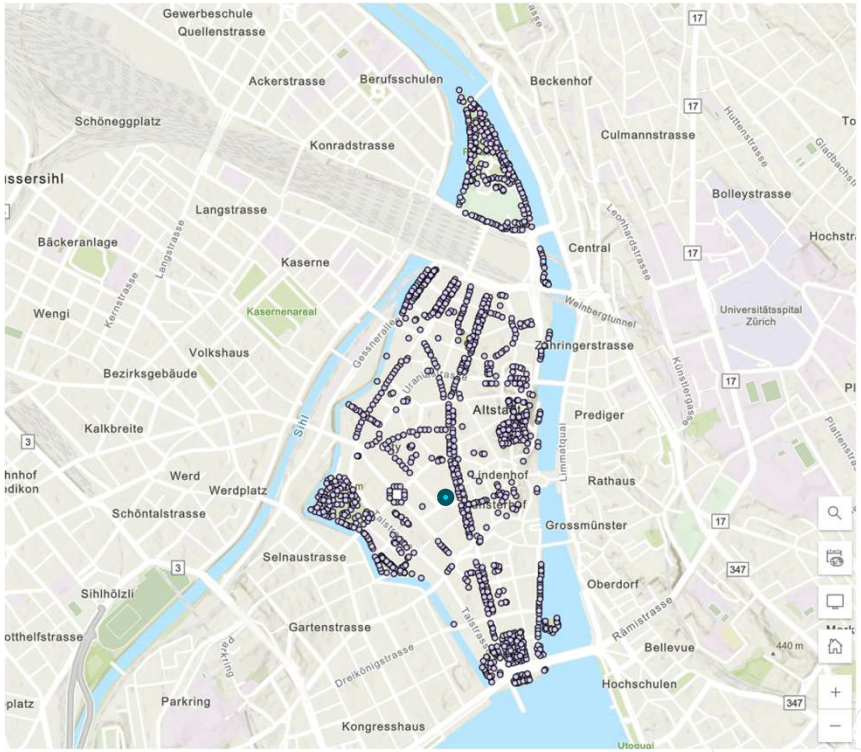
You are given a set of PointGroup about the trees in Zurich, and you want to find if your tree of interest is found in the set.

Linear search algorithm

```
for each point p in points {
     if p == p0
          return p
}
```

Is it correct? Any possibilities of not working as planned?
Is it efficient? How much is the computational load?

Efficiency = O(n)



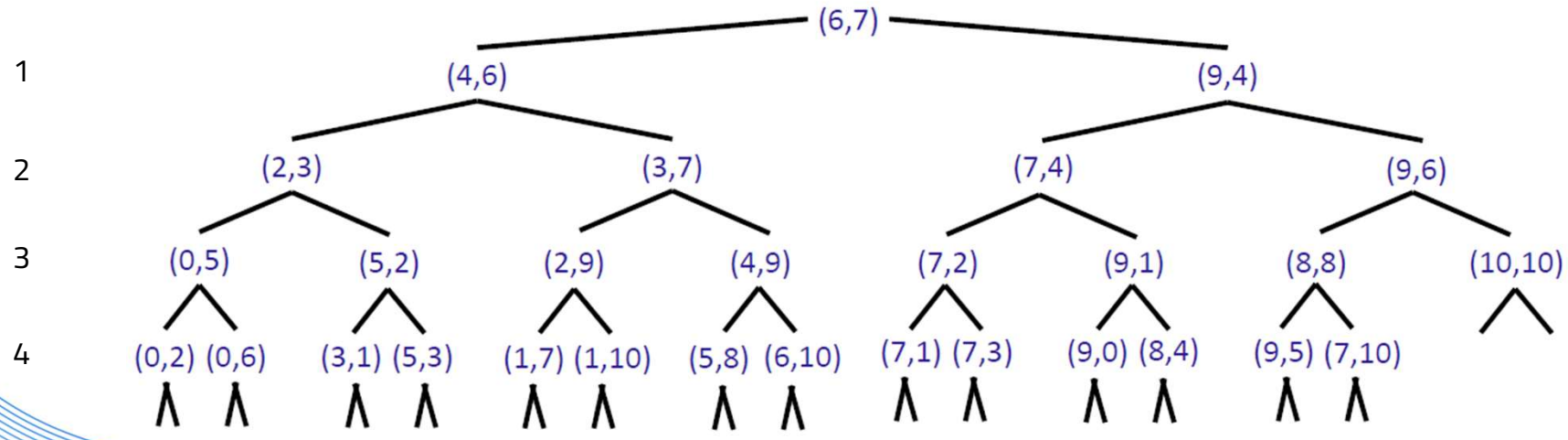
https://www.arcgis.com/home/item.html?id=8868a283da884257bfc032c730a80685; original data comes from Open Data Zürich

GEO876 recap:

Looking for a tree of interest

Binary tree approach (the case of balanced tree)

- Here, the key to success is to build the binary tree.
- At odd-numbered rows, we take two other coordinates and see their X; when it is smaller or equal, it goes to left; otherwise, it goes to the right
- At even-numbered rows, we see Y values; and do the same



GEO876 recap:

Looking for a tree of interest

Binary tree approach (the case of balanced tree)

```
p0 is the point we are looking for
t = root

while t is not null
    p = point at t
    if p == p0
        return p
    if level == even
        coordP, coordP0 = xCoord of p and p0
    if level == odd
        coordP, coordP0 = yCoord of p and p0
    if coordp0 is smaller or equal to coordP
        t = left branch of t
    else
        t = right branch of t
```

Is it correct? Any possibilities of not working as planned?
Is it efficient? How much is the computational load?

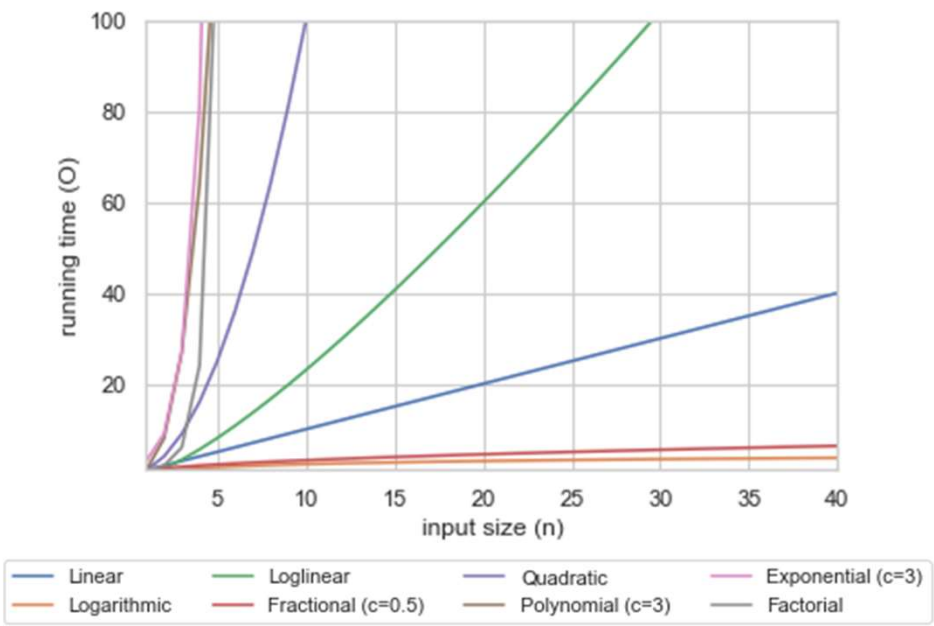
When the tree is balanced, the efficiency of running this for n points is determined by the height of tree (H).
Since the balanced tree can store a maximum number of n points

$2^0 + 2^1 + 2^2 + 2^3 + \ldots 2^H = 2^{H+1} - 1 = n$

So the height H = $\log_2(n+1)$-1 ≒ $\log_2(n)$
Efficiency = O($\log_2(n)$)

GEO876 recap:

Looking for a tree of interest



Linear search

Efficiency = O(n)

Binary tree approach (balanced)

Efficiency = $O(\log_2(n))$

GE0876 recap:

Generate a bounding box for a point group

You are given a set of PointGroup that represents trees in the city of Zurich.

How did you generate a bounding box, without using any off-the-shelf packages?

https://www.arcgis.com/home/item.html?id=8868a283da884257bfc032c730a80685; original data comes from Open Data Zürich

GEO876 recap:

Generate a bounding box for a point group

```
from dataclasses import dataclass
from typing import List

class Point:
    x: float
    y: float

class BoundingBox:
    min_x: float
    max_x: float
    min_y: float
    max_y: float
```

```
def bounding_box(points: List[Point]) -> BoundingBox:
    if not points:
        raise ValueError("empty point list")

    min_x = points[0].x
    max_x = points[0].x
    min_y = points[0].y
    max_y = points[0].y

    for p in points[1:]:
        if p.x < min_x:
            min_x = p.x
        if p.x > max_x:
            max_x = p.x
        if p.y < min_y:
            min_y = p.y
        if p.y > max_y:
            max_y = p.y

    return BoundingBox(min_x, max_x, min_y, max_y)
```

Is it correct? Any possibilities of not working as planned?
Is it efficient? How much is the computational load?

GEO876 recap:

Generate a bounding box for a point group

```
from dataclasses import dataclass
from typing import List

class Point:
    x: float
    y: float

class BoundingBox:
    min_x: float
    max_x: float
    min_y: float
    max_y: float
```

```
def bounding_box(points: List[Point]) -> BoundingBox:
    if not points:
        raise ValueError("empty point list")

    min_lat = min(p.lat for p in points)
    max_lat = max(p.lat for p in points)
    min_lon = min(p.lon for p in points)
    max_lon = max(p.lon for p in points)

    return BoundingBox(min_lat, max_lat, min_lon, max_lon)
```

Is it correct? Any possibilities of not working as planned?
Is it efficient? How much is the computational load?

Until next time

- Join the Teams: blyicls
- Browse the papers in Teams (they will be uploaded shortly)
- Sign yourself up to three papers with preference (1st,2nd,3rd) on Excel sheet

* For those who are in the “waitlist”, please come & talk to me